

Value Iteration

Solving a known MDP by Dynamic Programming

Seung Ho Go

July 26, 2026

Table of contents

- 1 Where we are: Dynamic Programming
- 2 The Value Iteration algorithm
- 3 Worked example: the robot car
- 4 Properties, limits, and Q-values

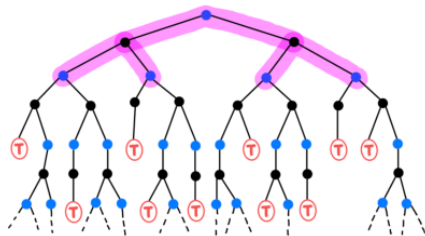
Recap: what are we trying to do?

Both **Dynamic Programming** and **Reinforcement Learning** exist to solve an MDP — and “solving” an MDP means **finding the optimal policy** π_* .

DP additionally requires the MDP to be **model-based**:

we know $p(s', r | s, a)$ and the rewards.

Knowing the model lets us compute expectations *exactly*, using **every** successor state — a **full backup**.



DP full backup: one update uses *all* branches.

Today: **Value Iteration**, the first of the two classic DP algorithms.

Two DP algorithms, two Bellman equations

Value Iteration (today)

Uses the **Bellman optimality equation**

$$v_*(s) = \max_a \sum p[r + \gamma v_*(s')]$$

Note the $\max_a$.

Policy Iteration (next time)

Uses the **Bellman expectation equation**

$$v_\pi(s) = \sum_a \pi(a|s) \sum p[r + \gamma v_\pi(s')]$$

Evaluate a fixed π , then improve it.

That single difference — **max over actions** vs. **average under π** — is what separates the two methods.

From equation to iteration

We want the **optimal state-value function** v_* , which satisfies

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')].$$

- v_* appears on *both* sides — and it must hold for **every** state simultaneously. Solving this system directly is infeasible.

Idea: turn the equation into a recurrence

Introduce an **estimator** V_k of v_* and iterate:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')]$$

If $V_k \rightarrow V^*$, the recurrence becomes an *equality* — which is exactly the Bellman optimality equation. So the limit **is** v_* .

Value Iteration in three steps

- 1 **Initialize** $V_0(s) = 0$ for all s (or randomly).

Terminal states are **always** fixed at $V(\text{terminal}) = 0$: no more actions, hence no more return.

- 2 **Iterate** the recurrence over all states until **convergence** to V^* (a full backup each sweep).
 - *synchronous*: compute V_{k+1} for all s , then update at once
 - *asynchronous*: update each s immediately as it is computed

- 3 **Extract the policy** by a **one-step lookahead**:

$$\pi_*(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V^*(s')]$$

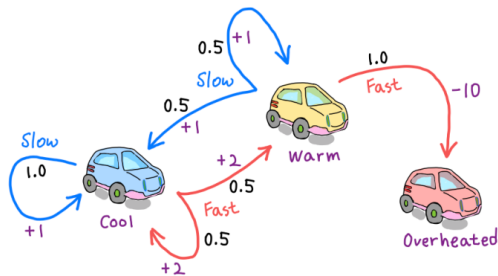
Step 2 costs almost everything; step 3 runs once at the end.

The robot car MDP

A robot car wants to travel **far and quickly**.

- **states:** cool, warm, **overheated** (terminal)
- **actions:** slow, fast
- **rewards:** slow = 1, fast = 2, but **-10** when it overheats
- $\gamma = 1$ (fixed, to keep arithmetic simple)

Transition probabilities are known $\Rightarrow$ **model-based**,
so DP applies.



Driving fast pays more — until the engine overheats and the game ends.

One update by hand: $V_2(\text{warm})$

Apply the recurrence at $s = \text{warm}$, using the already-computed V_1 (cool = 2, warm = 1, over = 0). Evaluate *both* actions, then take the max.

$$V_2(\text{Warm}) = \max_a \left\{ \begin{array}{l} 0.5(1+2) + 0.5(1+1) \\ 1.0(-10+0) \end{array} \right\}$$

← slow
← fast

slow: two successors, each w.p. 0.5, reward 1

$$0.5(1+2) + 0.5(1+1) = \mathbf{2.5}$$

fast: certain overhear, reward -10 , $V(\text{over}) = 0$

$$1.0(-10+0) = -10$$

$V_2(\text{warm}) = \max\{2.5, -10\} = \mathbf{2.5}$, so the best action at *warm* is **slow**.

Sweeping the table: $V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow \dots$

$V(s)$	cool	warm	over
V_2	3.5	2.5	0
V_1	2	1	0
V_0	0	0	0

Each row is one iteration over *all* states. Once V_{k+1} is complete, V_k can be discarded.

The policy settles early

In every iteration: **warm** $\rightarrow$ **slow** (avoid the -10), **cool** $\rightarrow$ **fast** (higher reward, no risk yet).

The *values* keep changing, but the **arg max** **stopped changing long ago**.

Remember this observation — it is Value Iteration's main weakness.

Synchronous vs. asynchronous backups

Synchronous

Compute $V_{k+1}(s)$ for **all** states from the old table V_k , then swap the whole table in at once.

Asynchronous

Update each state **immediately**. Later states in the same sweep then use the **fresher** values — usually more accurate.

Both are valid; asynchronous simply reuses newer information as soon as it exists.

Two disadvantages of Value Iteration

1. The policy converges long before the values do

The action attaining the max **rarely changes**. We saw it in the robot car: π was fixed from the very first sweeps, yet we keep iterating to polish V . $\Rightarrow$ **wasted computation**.

2. Each iteration is expensive

One sweep touches every state, every action, and every successor state:

$O(S^2A)$ per iteration — and many iterations are needed.

Convergence criterion. Fix a small ϵ (e.g. 10^{-4}) and stop when $|V_{k+1}(s) - V_k(s)| < \epsilon$ for **all** s .

Policy Iteration (*next lecture*) is designed to fix disadvantage 1.

Pseudocode

Value Iteration for estimating $\pi \approx \pi_*$

Hyperparameter: small threshold $\epsilon > 0$ for the convergence check

Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$, except $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

// old value

$V(s) \leftarrow \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$

// one backup

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

// largest change this sweep

Until $\Delta < \epsilon$

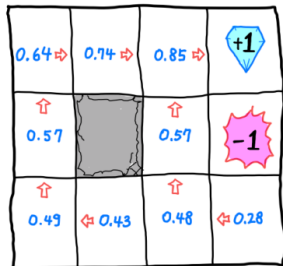
Output a deterministic policy $\pi \approx \pi_*$ such that

$\pi(s) = \arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$

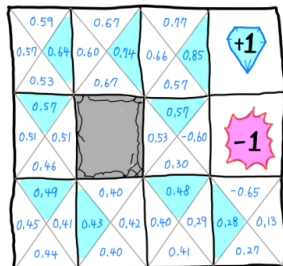
Δ is the largest change over all states — it is how we test convergence.

Grid world: values vs. Q-values

$V(s)$ after 100 iters



$Q(s, a)$ after 100 iters



noise = 0.2, $\gamma = 0.9$, living reward $c = 0$

From V^* we still need a **one-step lookahead**:

$$\pi_*(s) = \arg \max_a \sum p[r + \gamma V^*(s')]$$

From Q^* we just read off the best entry:

$$\pi_*(s) = \arg \max_a Q^*(s, a)$$

Actions are easier to select from Q-values than from values.

The Q-table — and why RL needs it

Q-table

actions

	↑↑	↓↓	←←	→→
1	0.49	0.44	0.45	0.41
2	0.40	0.40	0.43	0.42
3	0.48	0.41	0.40	0.29
⋮	⋮	⋮	⋮	⋮
9	0.77	0.57	0.66	0.85

states

Q-values after 100 iterations

A **Q-table** stores one value per **(state, action)** pair: 9 states $\times$ 4 actions here.

Trade-off

- V : table of size $|S|$ — cheaper
- Q : table of size $|S| \times |A|$ — bigger, but the policy is immediate

Why DP prefers V , but RL needs Q

Extracting π_* from V^* requires the **transition probabilities**. DP has them; **model-free RL does not** — so RL learns Q directly.

Takeaways

- ① **Value Iteration** turns the Bellman **optimality** equation into a recurrence and iterates V_k until it converges to V^* .
- ② Three steps: **initialize** $\rightarrow$ **iterate to V^*** $\rightarrow$ **one-step lookahead** for π_* .
- ③ It is **model-based** and uses **full backups** (every successor state).
- ④ Two weaknesses: the **policy converges long before the values**, and each sweep costs $O(S^2A)$.
- ⑤ Q^* makes policy extraction trivial — which is exactly why **model-free RL uses Q-tables**.

Next: **Policy Iteration** — evaluate, improve, repeat; and it fixes weakness 1.

Thank you

Questions?